

MISE EN PLACE D'UN SYSTEME DECISIONNEL POUR LE PILOTAGE DE L'ACTIVITE DE CREDIT ET ANALYSE DU RISQUE BANCAIRE



Réalisé par :

- Audes Mariana MOUSSAVOU MOUSSAVOU
- Stécy MOMBO AZZILE
- Awa aimée BENZEKRY
- Ramatoulaye DIAWANE

Encadré par :

Mme ABOUREZQ

TABLE DES MATIERES

INTRODUCTION.....	4
CHAPITRE 1 : MODELISATION ET GENERATION DES DONNEES SOURCES	5
1. Matrice Fait/Dimension.....	5
A. Montant Initial (Somme).....	6
B. Taux d'Intérêt (Moyenne)	6
C. Montant Remboursé (Somme)	6
D. Capital Restant Dû (CRD).....	6
E. Jours de Retard	6
2. JUSTIFICATION DU CROISEMENT MULTIDIMENSIONNEL	7
A. L'importance de la Dimension Temps	7
B. L'importance de la Dimension Client.....	7
C. L'importance de la Dimension Agence.....	7
D. L'importance de la Dimension Offre	7
3. Stratégie de Simulation et Génération des Données (Python)	7
A. Stack Technique utilisée	7
B. Logique de génération des tables	8
C. Injection de l'Intelligence Métier.....	9
CHAPITRE 2 : AUTOMATISATION DE L'INTEGRATION DE DONNEES (ETL)	12
1. Environnement de Stockage : SQL Server	12
2. Architecture du package SSIS	13
A. Le Flux de Contrôle (Control Flow)	13
B. Le Flux de Données (Data Flow)	14
CHAPITRE 3 : RESTITUTION ET ANALYSE DECISIONNELLE (POWER BI)	16
1. Architecture du Modèle de Données (Power BI)	16
2. Analyse des Visuels et Interprétations Métier	17
A. Cartographie de la Production.....	17
B. Analyse de Corrélation Risque/Taux.....	17
C. Somme des retards / CSP	18
CONCLUSION	19

TABLE DES FIGURES

Figure 1: Stack Technique utilisée	8
Figure 2: Génération des tables de dimensions	8
Figure 3: Génération de la Table de Faits	9
Figure 4: Injection de l'Intelligence Métier	10
Figure 5: les fichiers créés après exécution de nos codes python	10
Figure 6: Script de création de la base de données	12
Figure 7: schéma (DDL)	13
Figure 8: chargement des dimensions	13
Figure 9: Intégration finale	14
Figure 10: Flux de données	15
Figure 11: Schéma en étoile sur Power BI	16
Figure 12: Analyse Géographique	17
Figure 13: Statu par taux d'intérêt	17

TABLE DES TABLEAUX

Tableau 1: Table Fait/Dim	6
---------------------------------	---

INTRODUCTION

Dans un secteur financier de plus en plus numériser, la gestion des données est devenue un levier stratégique majeur. Pour une institution bancaire, la capacité à suivre en temps réel la santé de son portefeuille de prêts et à anticiper les risques d'impayés est une question de pérennité.

Cependant, les banques font souvent face à des volumes de données massifs et hétérogènes, rendant difficile une vision claire de la performance des agences ou du profil de risque des clients. Comment transformer ces données brutes en indicateurs de performance (KPIs) fiables et interactifs pour aider les décideurs ?

L'objectif de ce projet est de concevoir et réaliser une solution de Business Intelligence (BI) de bout en bout. Ce travail couvre l'intégralité du cycle de vie de la donnée : de la simulation d'un dataset bancaire réaliste via Python, à la modélisation d'un entrepôt de données sous SQL Server, en passant par l'automatisation des flux ETL avec SSIS, pour aboutir à un tableau de bord décisionnel interactif sous Power BI.

Ce rapport s'articule autour de trois parties majeures :

- La première partie détaillera la modélisation et la génération des données sources.
- La deuxième partie présentera la mise en place de la chaîne ETL pour le stockage structuré.
- Enfin, la troisième partie sera consacrée à l'analyse visuelle et à l'interprétation des résultats métier

CHAPITRE 1 : MODELISATION ET GENERATION DES DONNEES SOURCES

Avant de concevoir un système décisionnel performant, il est impératif de définir avec précision la structure des données qui alimentera les analyses. Ce premier chapitre constitue la phase de conception et de préparation de notre projet de Business Intelligence.

L'enjeu est double : d'une part, transformer les besoins métiers liés à la gestion des crédits bancaires en une architecture de données optimisée (le Schéma en Étoile), et d'autre part, pallier l'absence de données réelles par une simulation informatique d'haute-fidélité.

Nous détaillerons d'abord la méthodologie de Kimball utilisée pour identifier nos indicateurs clés de performance (KPIs) et nos axes d'analyse. Ensuite, nous présenterons le processus de génération de données via Python, en expliquant comment nous avons injecté des règles métier spécifiques au secteur bancaire pour garantir la cohérence et la richesse des futures visualisations sous Power BI

1. Matrice Fait/Dimension

L'objectif premier de ce projet est de fournir aux décideurs bancaires une vision claire du portefeuille de crédits. Pour ce faire, nous avons défini une Matrice Fait/Dimension, identifiant les processus métier à analyser (les Faits) et les axes d'analyse (les Dimensions).

Le Processus Métier : L'octroi des crédits et le suivi des remboursements de prêts.

Les Indicateurs (Faits) : Montant initial, Capital restant dû, Jours de retard, Montant remboursé.

Les Axes (Dimensions) : Temps (quand ?), Clients (qui ?), Agences (où ?), Offres (quel produit ?).

Processus Métier / Indicateurs	DIM_TEMPS	DIM_CLIENTS	DIM_AGENCES	DIM_OFFRES	Nature de l'indicateur
Montant Initial	X	X	X	X	Mesure (Volume)
Montant Remboursé	X	X	X	X	Mesure (Performance)
Capital Restant Dû	X	X	X	X	Mesure (Trésorerie)
Jours de Retard	X	X	X	X	Mesure (Risque)

Taux d'Intérêt	X	X	X	X	Mesure (Rentabilité)
----------------	---	---	---	---	----------------------

Tableau 1: Table Fait/Dim

Pour piloter l'activité de crédit, nous avons sélectionné des indicateurs clés répartis en trois catégories : Volume, Performance et Risque.

A. Montant Initial (Somme)

Représente le capital total prêté par la banque sur une période donnée. C'est l'indicateur principal de la force commerciale et de la croissance de l'établissement.

B. Taux d'Intérêt (Moyenne)

Mesure la rentabilité brute des crédits. Il permet d'analyser si la banque applique des taux cohérents avec le profil de risque du client.

C. Montant Remboursé (Somme)

Indique le cumul des fonds déjà récupérés par la banque (capital + intérêts). C'est le flux de trésorerie entrant (Cash-Flow).

D. Capital Restant Dû (CRD)

Représente l'exposition financière actuelle de la banque. C'est le montant total que les clients doivent encore rembourser à l'instant T.

E. Jours de Retard

C'est l'indicateur d'alerte le plus critique. Il mesure le nombre de jours écoulés depuis la date d'échéance impayée.

Exemple :

- 0 jour : Dossier sain.
- 1 à 90 jours : Dossier en retard (Relances).

2. JUSTIFICATION DU CROISEMENT MULTIDIMENSIONNEL

Le choix de relier chaque dimension à l'ensemble des indicateurs répond à un besoin de pilotage à 360°. Cette structure permet de "découper" (Slicing) les données selon n'importe quel angle pour obtenir des réponses précises.

A. L'importance de la Dimension Temps

Pourquoi ? Pour analyser la saisonnalité et les tendances.

Exemple : "Est-ce que le Montant Initial (production) est plus élevé en décembre ?" ou "Les Jours de Retard augmentent-ils après les périodes de vacances ?"

B. L'importance de la Dimension Client

Pourquoi ? Pour la segmentation et le profilage.

Exemple : "Quelle Catégorie Socio-Professionnelle (CSP) présente le plus de Capital Restant Dû ?" ou "Le Score de Confiance est-il réellement corrélé à la baisse des Retards ?"

C. L'importance de la Dimension Agence

Pourquoi ? Pour le contrôle de performance géographique.

Exemple : "Quelle Région affiche le meilleur Taux de Remboursement ?" ou "Quelle agence est-elle plus exposée au risque ?"

D. L'importance de la Dimension Offre

Pourquoi ? Pour la stratégie produit.

Exemple : "Le Prêt Immobilier est-il plus rentable (via le Taux d'Intérêt) que le Prêt Automobile ?" ou "Quelles offres génèrent le plus de dossiers en Contentieux ?"

3. Stratégie de Simulation et Génération des Données (Python)

L'indisponibilité de données bancaires réelles, soumises au secret professionnel, a nécessité le développement d'un moteur de simulation sur mesure en Python. L'objectif n'était pas seulement de créer des chiffres, mais de simuler un écosystème bancaire cohérent.

A. Stack Technique utilisée

Pour cette phase, trois bibliothèques majeures ont été mobilisées :

- Pandas : Pour la structuration des données sous forme de tables (DataFrames) et l'export au format CSV.

- Faker : Pour générer des données d'identité réalistes (noms, prénoms, villes) adaptées au contexte géographique.
- Numpy & Random : Pour l'application de lois de probabilité sur les montants et les statuts de crédit.

```
C: > Users > 22891 > Desktop > Projet BI > Projet_BI.py > ...
1 import pandas as pd
2 import numpy as np
3 from faker import Faker
```

Figure 1: Stack Technique utilisée

B. Logique de génération des tables

Comme illustré dans l'extrait de code ci-dessous, le processus suit une hiérarchie stricte pour garantir l'intégrité référentielle :

- Génération des Dimensions :

Création d'un pool de 1000 clients avec des profils variés (Âge, CSP, Score de confiance).

Configuration de 10 agences réparties par régions.

Définition du catalogue de 4 produits financiers (Immobilier, Auto, Conso, Étudiant).

```
# ---- CONFIGURATION ----
NB_CLIENTS = 1000
NB_PRETS = 3000
DATE_DEBUT = datetime(2020, 1, 1)
DATE_FIN = datetime(2025, 1, 1)

# ---- 1. DIMENSION AGENCES ----
regions = ['Île-de-France', 'Auvergne-Rhône-Alpes', 'Provence-Alpes-Côte d\'Azur', 'Hauts-de-France']
agences_data = []
for i in range(1, 11):
    agences_data.append({
        'Nom_Agence': f"Agence {fake.city()}",
        'Region': random.choice(regions),
        'Type_Agence': random.choice(['Principale', 'Secondaire'])
    })
df_agences = pd.DataFrame(agences_data)

# ---- 2. DIMENSION OFFRES ----
offres_data = [
    {'ID_Offre': 1, 'Type_Pret': 'Immobilier', 'Taux_Base': 1.8},
    {'ID_Offre': 2, 'Type_Pret': 'Automobile', 'Taux_Base': 3.5},
    {'ID_Offre': 3, 'Type_Pret': 'Consommation', 'Taux_Base': 5.2},
    {'ID_Offre': 4, 'Type_Pret': 'Étudiant', 'Taux_Base': 0.9}
]
df_offres = pd.DataFrame(offres_data)

# ---- 3. DIMENSION CLIENTS ----
clients_data = []
csp_liste = ['Salarié', 'Cadre', 'Indépendant', 'Retraité', 'Étudiant']
for i in range(1, NB_CLIENTS + 1):
    age = random.randint(18, 80)
    clients_data.append({
        'ID_Client': i,
        'Nom': fake.last_name(),
        'Age': age,
        'CSP': random.choice(csp_liste),
        'Anciennete_Mois': random.randint(0, 120),
        'Score_Confiance': random.randint(1, 5) # 1: Risqué, 5: Excellent
    })
df_clients = pd.DataFrame(clients_data)
```

Figure 2: Génération des tables de dimensions

- Génération de la Table de Faits :

Chaque ligne de fait "tire au sort" un ID client, un ID agence et un ID offre existants, simulant ainsi une transaction réelle.


```
# ---- 4. TABLE DE FAITS (CREDITS) ----
faits_data = []
statuts = ['Actif', 'Soldé', 'Retard', 'Contentieux']

for i in range(1, NB_PRETS + 1):
    client = df_clients.sample(1).iloc[0]
    offre = df_offres.sample(1).iloc[0]
    agence = df_agences.sample(1).iloc[0]

    # Logique de montant selon le type de prêt
    if offre['Type_Pret'] == 'Immobilier':
        montant = random.randint(100000, 500000)
        duree = random.choice([120, 180, 240, 300])
    else:
        montant = random.randint(2000, 50000)
        duree = random.choice([12, 24, 36, 48, 60])

    # Date de signature cohérente
    date_signature = fake.date_between(start_date=DATE_DEBUT, end_date=DATE_FIN)
```

Figure 3: Génération de la Table de Faits

C. Injection de l'Intelligence Métier

Pour rendre l'analyse Power BI pertinente, le script intègre des règles métier :

Cohérence Financière : Le Capital Restant Dû est calculé dynamiquement :

Simulation du Risque : Le statut "Retard" ou "Contentieux" n'est pas attribué au hasard. Il est pondéré pour refléter la réalité d'un portefeuille bancaire sain.

Variabilité des Taux : Le taux d'intérêt final est corrélé au score de confiance du client, simulant une négociation bancaire réelle.

```

statut = np.random.choice(statuts, p=[0.7, 0.2, 0.07, 0.03])

if statut == 'Soldé':
    montant_rembourse = montant
    jours_retard = 0
elif statut == 'Retard':
    montant_rembourse = round(montant * random.uniform(0.1, 0.8), 2)
    jours_retard = random.randint(5, 90)
elif statut == 'Contentieux':
    montant_rembourse = round(montant * random.uniform(0, 0.3), 2)
    jours_retard = random.randint(91, 365)
else: # Actif
    montant_rembourse = round(montant * random.uniform(0.05, 0.95), 2)
    jours_retard = 0

faits_data.append({
    'ID_Pret': i,
    'ID_Client': client['ID_Client'],
    'ID_Agence': agence['ID_Agence'],
    'ID_Offre': offre['ID_Offre'],
    'Date_Signature': date_signature,
    'Montant_Initial': montant,
    'Duree_Mois': duree,
    'Taux_Interet': offre['Taux_Base'] + (6 - client['Score_Confiance']) * 0.5,
    'Montant_Rembourse': montant_rembourse,
    'Capital_Restant_Du': round(montant - montant_rembourse, 2),
    'Jours_Retard': jours_retard,
    'Statut': statut
})

```

Figure 4: Injection de l'Intelligence Métier

En conclusion, ce premier chapitre a permis de poser les jalons fondamentaux de notre projet décisionnel. À travers une analyse rigoureuse des besoins métiers liés au crédit bancaire, nous avons conçu une architecture en étoile robuste, centrée sur une table de faits unique regroupant l'ensemble des indicateurs de performance (Volume, Recouvrement, Risque).

La phase de simulation en Python a constitué une étape clé : elle nous a permis de générer un jeu de données de 3000 transactions et 1000 clients d'un grand réalisme.

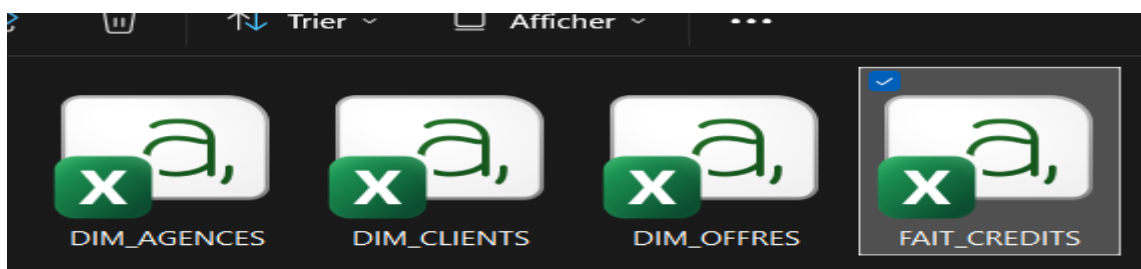


Figure 5: les fichiers créés après exécution de nos codes python

Comme illustré par l'algorithme d'injection de l'intelligence métier, chaque donnée respecte des règles de gestion bancaire strictes (calcul du taux d'intérêt selon le score de confiance, gestion du capital restant dû et pondération des retards).

Désormais, nous disposons de fichiers sources cohérents et structurés en format CSV. Cependant, ces données sont encore isolées du système d'information. La problématique suivante consiste donc à automatiser leur intégration et leur stockage sécurisé dans un entrepôt de données. C'est ce que nous aborderons dans le Chapitre 2, consacré à la mise en œuvre de la chaîne ETL avec SSIS.

CHAPITRE 2 : AUTOMATISATION DE L'INTEGRATION DE DONNEES (ETL)

Une fois la phase de modélisation et de génération des données sources achevée, l'étape suivante consiste à orchestrer le transfert de ces informations vers un entrepôt de données centralisé. Ce processus, appelé ETL (Extract, Transform, Load), est le pilier technique de toute solution décisionnelle.

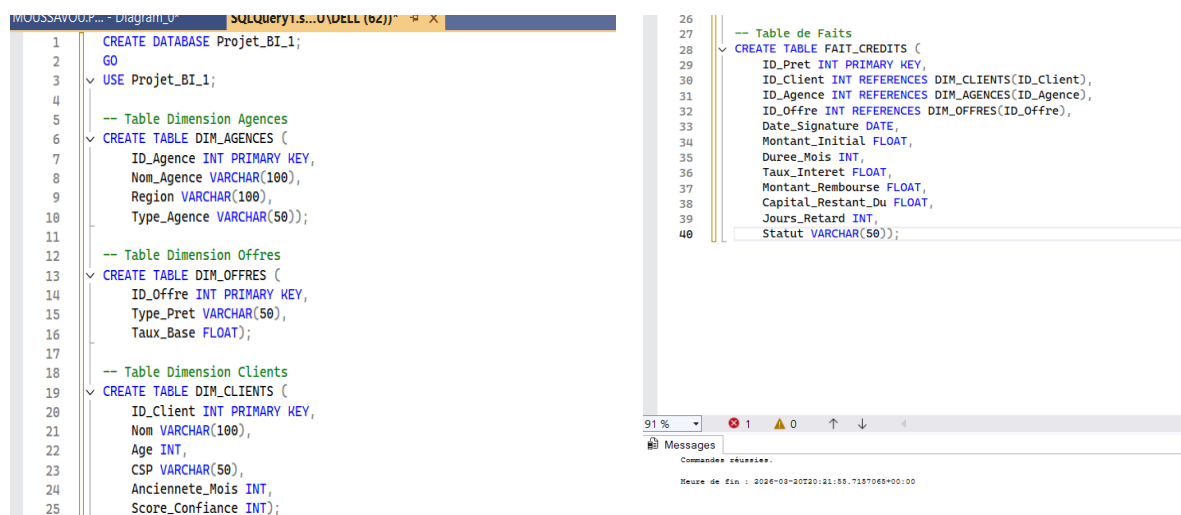
Dans ce chapitre, nous détaillerons l'utilisation de la suite Microsoft, notamment SQL Server Integration Services (SSIS), pour automatiser l'intégration de nos fichiers CSV dans une base de données SQL Server. Nous aborderons la configuration de l'environnement, la conception des flux de données et les solutions apportées aux défis techniques rencontrés lors de l'intégration

1. Environnement de Stockage : SQL Server

Avant de procéder à l'intégration, nous avons configuré le réceptacle des données sous SQL Server Management Studio (SSMS).

Grâce à SSMS nous avons pu :

- Créer la base de données : `Projet_BI_1`.



```
1 CREATE DATABASE Projet_BI_1;
2 GO
3 USE Projet_BI_1;
4
5 -- Table Dimension Agences
6 CREATE TABLE DIM_AGENCES (
7     ID_Agence INT PRIMARY KEY,
8     Nom_Agence VARCHAR(100),
9     Region VARCHAR(100),
10    Type_Agence VARCHAR(50));
11
12 -- Table Dimension Offres
13 CREATE TABLE DIM_OFFRES (
14     ID_Offre INT PRIMARY KEY,
15     Type_Pret VARCHAR(50),
16     Taux_Base FLOAT);
17
18 -- Table Dimension Clients
19 CREATE TABLE DIM_CLIENTS (
20     ID_Client INT PRIMARY KEY,
21     Nom VARCHAR(100),
22     Age INT,
23     CSP VARCHAR(50),
24     Anciennete_Mois INT,
25     Score_Confiance INT);
26
27 -- Table de Faits
28 CREATE TABLE FAIT_CREDITS (
29     ID_Pret INT PRIMARY KEY,
30     ID_Client INT REFERENCES DIM_CLIENTS(ID_Client),
31     ID_Agence INT REFERENCES DIM_AGENCES(ID_Agence),
32     ID_Offre INT REFERENCES DIM_OFFRES(ID_Offre),
33     Date_Signature DATE,
34     Montant_Initial FLOAT,
35     Duree_Mois INT,
36     Taux_Interet FLOAT,
37     Montant_Rembourse FLOAT,
38     Capital_Restant_Du FLOAT,
39     Jours_Retard INT,
40     Statut VARCHAR(50));
```

Figure 6: Script de création de la base de données

- Définir notre schéma (DDL) : Création des tables DIM_CLIENTS, DIM_AGENCES, DIM_OFFRES et FAIT_CREDITS



Figure 7: schéma (DDL)

2. Architecture du package SSIS

L'outil Visual Studio a été utilisé pour concevoir le package ETL. La logique a été divisée en deux niveaux :

A. Le Flux de Contrôle (Control Flow)

Il s'agit de définir l'ordonnancement des tâches. Pour respecter l'intégrité référentielle, nous avons mis en place une séquence stricte :

- Chargement des Dimensions : Intégration prioritaire des tables de dimension.

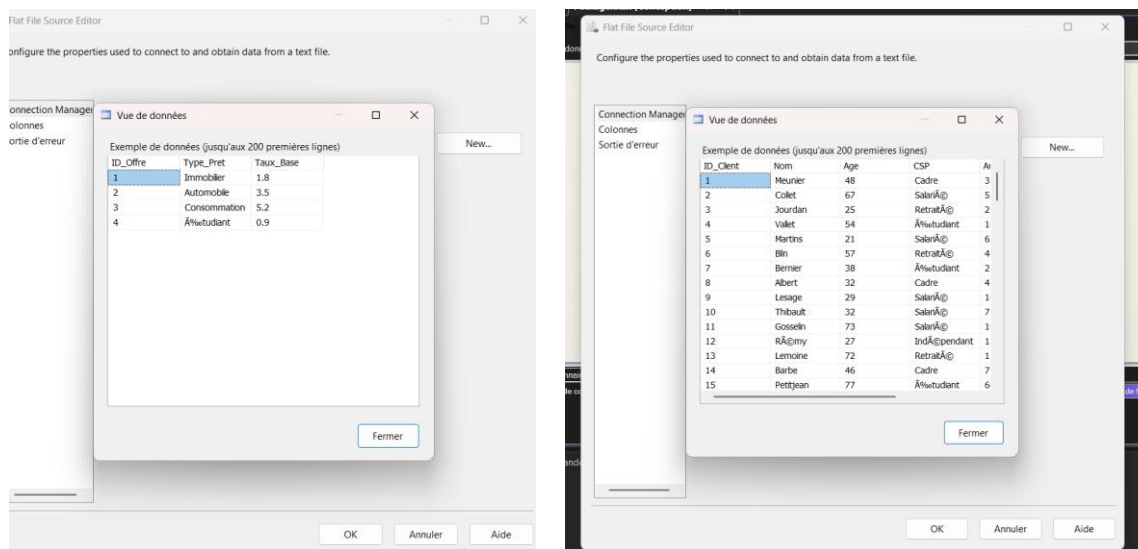


Figure 8: chargement des dimensions

- Chargement des Faits : Intégration finale de la table FAIT_CREDITS une fois les clés étrangères disponibles

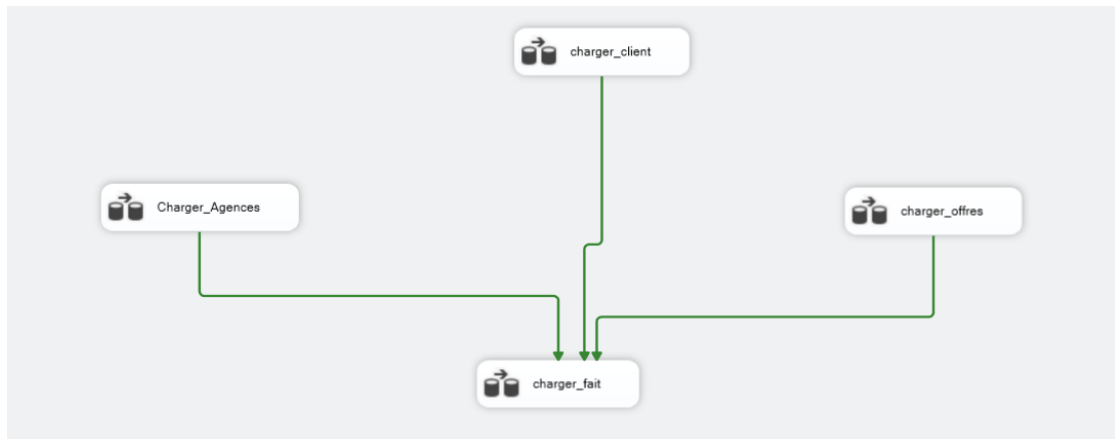
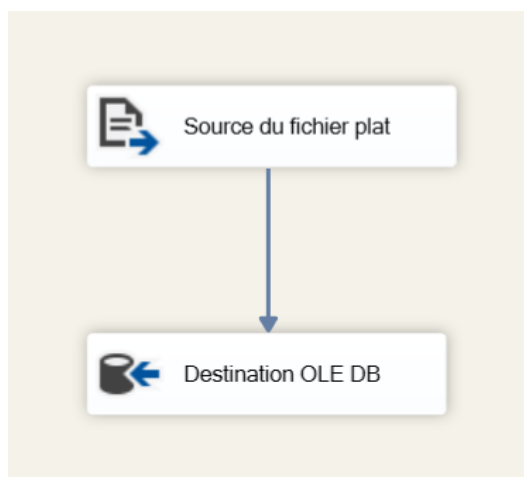
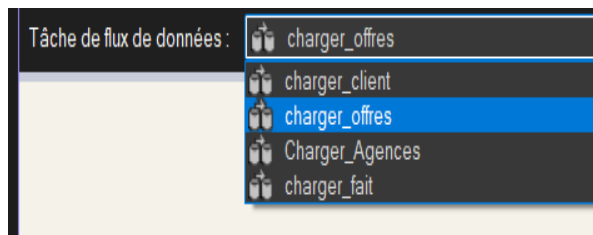


Figure 9: Intégration finale

B. Le Flux de Données (Data Flow)

Pour chaque table, une "tuyauterie" spécifique a été configurée :

- Source du fichier plat : Lecture et parsing des fichiers CSV.
- Destination OLE DB : Insertion des données vers l'instance SQL Server avec mapping des colonnes.



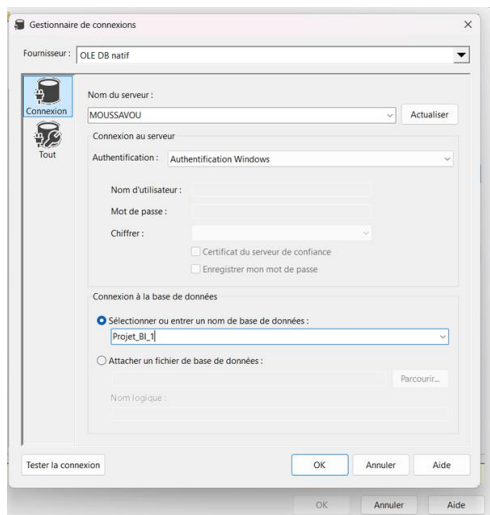
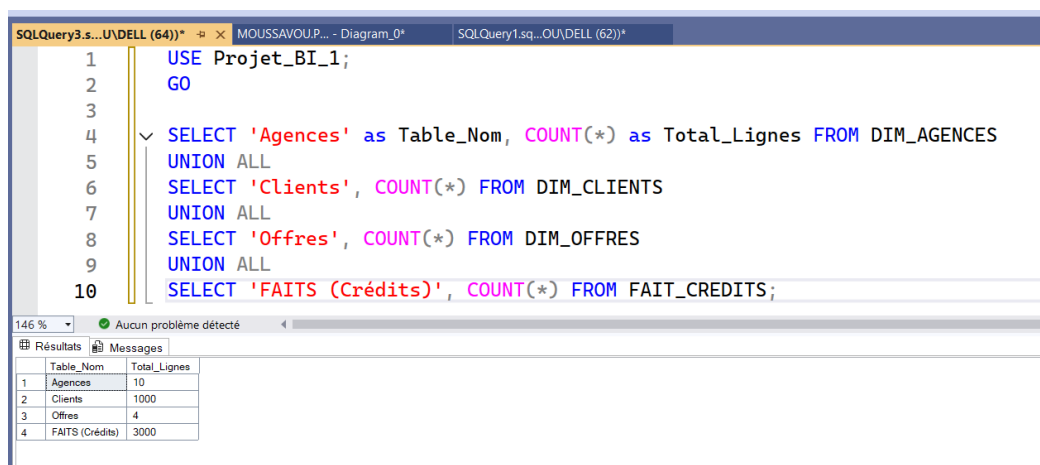


Figure 10: Flux de données

La mise en place de cette chaîne ETL sous SSIS garantit désormais une alimentation fiable et automatisée de notre Data Warehouse. Avec 3000 lignes de faits et 1000 clients chargés avec succès et sans erreurs de cohérence, l'entrepôt de données est désormais prêt pour l'exploitation analytique.



Bien que des solutions comme Pentaho aient été envisagées, le choix s'est porté sur la suite Microsoft (SSIS) pour garantir une homogénéité totale de la chaîne de valeur. L'intégration native entre SQL Server et Power BI permet une synchronisation plus fluide et une meilleure productivité, évitant ainsi les problèmes d'interopérabilité entre outils de différents éditeurs.

Le prochain chapitre sera consacré à la valorisation de ces données à travers la création d'un tableau de bord décisionnel sous Power BI.

CHAPITRE 3 : RESTITUTION ET ANALYSE DECISIONNELLE (POWER BI)

Ce dernier chapitre marque l'aboutissement du projet. Après avoir modélisé, simulé et intégré les données dans notre entrepôt SQL Server, nous procédons ici à leur valorisation visuelle. L'objectif est de transformer 3000 lignes de transactions en un tableau de bord interactif permettant un pilotage granulaire de l'activité de crédit et une surveillance accrue du risque client via Power BI.

1. Architecture du Modèle de Données (Power BI)

Dès l'importation des tables depuis SQL Server, Power BI a reconstitué notre Schéma en Étoile.

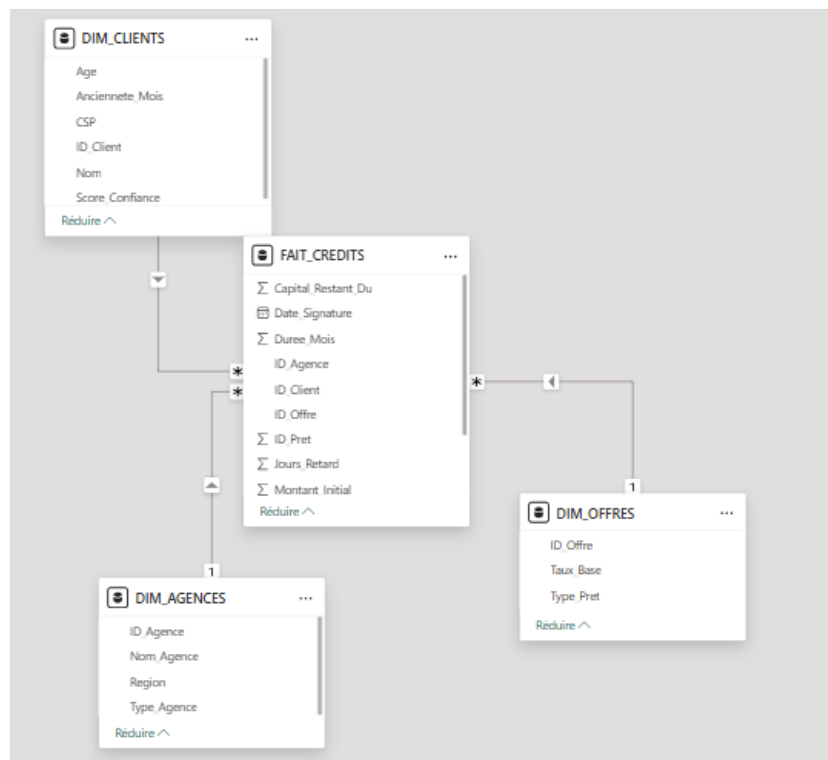


Figure 11: Schéma en étoile sur Power BI

Les Relations : Les tables de dimensions (Clients, Agences, Offres) sont liées à la table FAIT_CREDITS via des relations 1-à-Plusieurs (One-to-Many).

L'Intégrité : Cette structure garantit que tout filtre appliqué sur une dimension se répercute instantanément sur l'ensemble des indicateurs de la table de faits.

2. Analyse des Visuels et Interprétations Métier

A. Cartographie de la Production



Figure 12: Analyse Géographique

B. Analyse de Corrélation Risque/Taux

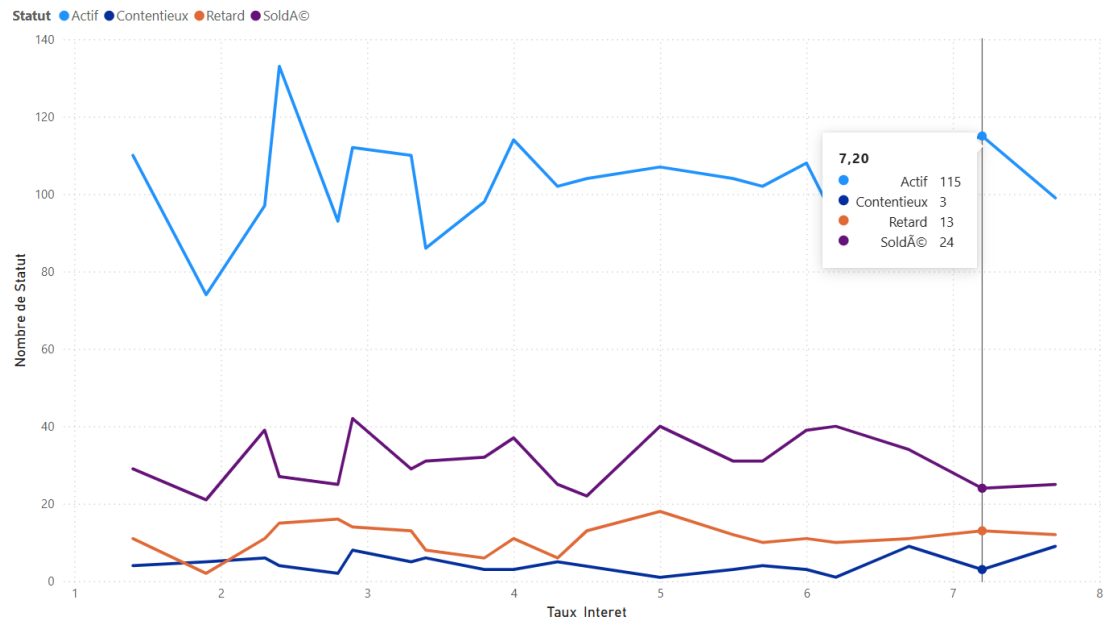


Figure 13: Statu par taux d'intérêt

C. Somme des retards / CSP



La mise en place de ce tableau de bord interactif offre une visibilité sans précédent sur le portefeuille de crédits. En combinant, la puissance de calcul de SQL Server et la souplesse visuelle de Power BI, nous avons créé un véritable outil d'aide à la décision, capable d'alerter les banquiers sur les zones de risques tout en célébrant les succès commerciaux.

CONCLUSION

Ce projet de Business Intelligence nous a permis de concevoir une solution décisionnelle complète, répondant aux enjeux critiques du pilotage de l'activité de crédit bancaire. À travers ce travail, nous avons démontré qu'il est possible de transformer des données brutes en un outil d'aide à la décision stratégique pour une institution financière.

Le parcours réalisé a couvert l'intégralité du cycle de vie de la donnée :

- La simulation métier via Python, qui a permis de créer un environnement de test réaliste et cohérent.
- La structuration technique sous SQL Server, avec une modélisation en étoile optimisée pour l'analyse multidimensionnelle.
- L'automatisation ETL avec SSIS, assurant une intégration fiable des données tout en gérant les défis de qualité et d'intégrité.
- La valorisation analytique sous Power BI, offrant une vision à 360° sur la performance commerciale et le risque d'impayés.

Au-delà de l'aspect technique, ce projet met en lumière l'importance de la Data Culture dans le secteur bancaire. Grâce aux indicateurs de retard et à la segmentation client par CSP, les gestionnaires peuvent désormais agir de manière proactive plutôt que réactive.

Perspectives d'évolution :

Ce système constitue un socle solide qui pourrait, à terme, être enrichi par l'intégration d'algorithmes de Machine Learning pour prédire les défauts de paiement avant qu'ils ne surviennent (Analyse Prédictive). Ce projet confirme que la maîtrise de la chaîne MSBI est un atout indispensable pour accompagner la transformation digitale des banques.